ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

PÓS-GRADUAÇÃO EM SISTEMAS COMPUTACIONAIS EMBEBIDOS (PGSCE)

*MÓDULO DE INTELIGÊNCIA ARTIFICIAL EM SISTEMAS EMBEBIDOS (*IARSEM*)*

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

UNIVERSIDADE TÉCNICA DO PORTO

Class Script:

Weka - Classification

Weka - Classificação de dados

Version Control

Version number	Date issued	Authors	Update information
V1.0	14 Set 2026	Veríssimo Santos (VMS)	Original version of the script.

Table of Contents

1. Introduction 1

1.1 Context 1

1.2 Motivation 1

1.3 Objective 1

1.4 Classification 1

1.4.1 Support Vector Machine 2

1.4.2 Artificial Neural Network 3

1.4.3 Decision Tree 4

1.4.4 Bagging 5

1.4.5 Random Forest 5

2. Weka 6

2.1 Key features 6

2.2 Setup 7

3. Tutorial example 7

4. Exercises 10

4.1 Problem A 10

4.2 Problem B 10

5. Challenge 10

6. References 11

1. Introduction

1.1 Context

The ubiquity of computation created a plenteous source of data that has grown in importance in our daily lives. Currently, we live in a data-driven world where data gathering, pre-processing, storage, extraction of knowledge and inference are constantly being automated and improved. A big part of this revolution has been propelled by machine learning which is a broad and multidisciplinary field that uses algorithms to enable computer systems to learn and improve from data without being explicitly programmed. Currently, machine learning is being applied to improve and support almost all fields of human knowledge.

1.2 Motivation

Classification is a fundamental machine-learning technique that is widely used to automate many tasks that involve some sort of classification done by or with human intervention. For example, email spam filtering, image recognition, and medical diagnosis are a few applications among many others. Classification models save significant time and resources and often can achieve higher accuracy and consistency than humans, especially in cases where the data is highly complex and/or there are many variables to consider.

1.3 Objective

This lab script aims to explore the concept and the application of classification to datasets using the Weka machine-learning tool.

1.4 Classification

Classification is a machine-learning technique that enables the automatic and accurate classification of data into different categories or classes. The goal is to assign to each input data instance (point) one or several predefined categories or classes based on the values of its feature set. The input data can be text, images, or numerical data and the number of classes varies from two (binary classification e.g. true/false) up to multi-class (e.g. animal species).

The process of classification involves the supervised training of a model on a labelled dataset, where the correct class labels are known for each input sample. The model uses training data to learn a decision boundary or function that maps input features to their corresponding class labels. Once the model has been trained, it can be used to predict the class labels of new, unseen data points.

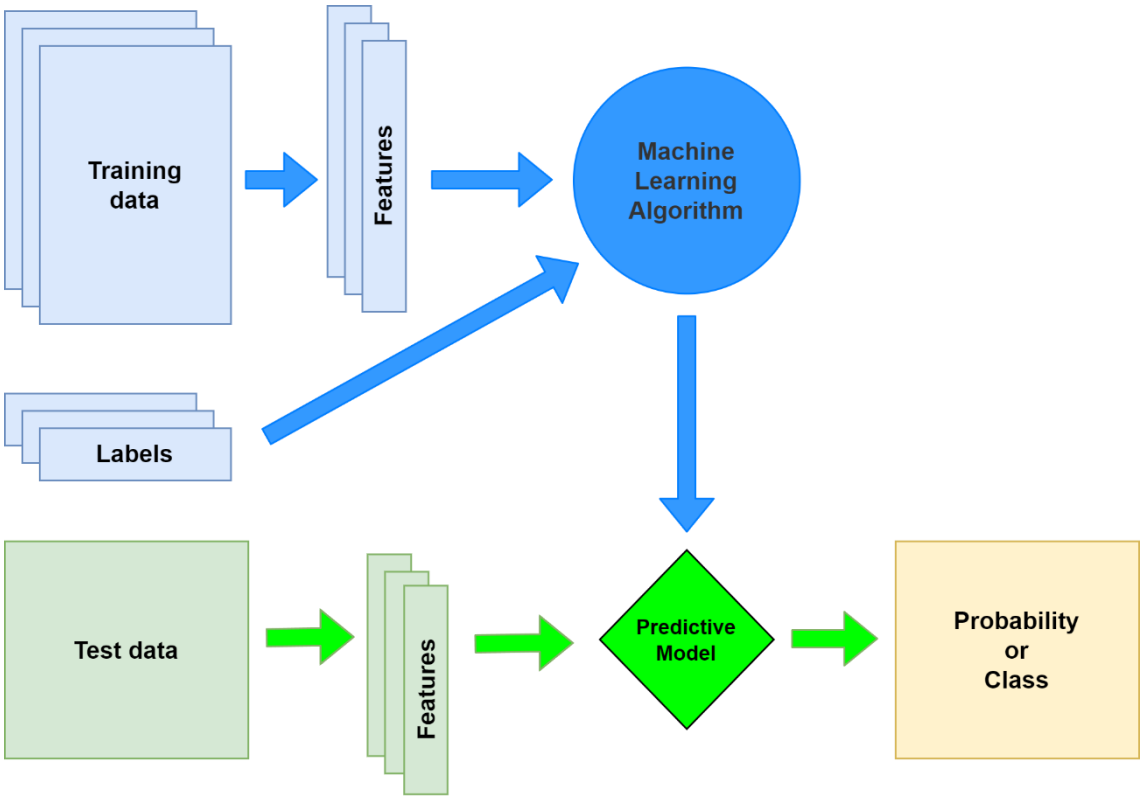


Figure 1 – Supervised Learning

There are many different classification algorithms available in machine learning, including logistic regression, decision trees, random forests, support vector machines (SVMs), and neural networks. Each with its strengths and weaknesses, and the choice of an algorithm often depends on the specific problem at hand and the nature of the input data.

Classification is widely used in spam filtering, sentiment analysis, image recognition, and medical diagnosis, among many others.

1.4.1 Support Vector Machine

A Support Vector Machine (SVM) is a type of machine learning algorithm that can be used for classification and regression analysis. SVMs work by finding the optimal hyperplane in a high-dimensional space that best separates the different classes' data points.

In simple terms, SVMs can be thought of as drawing a line or plane between data points of different classes to create a boundary that separates them. The SVM algorithm seeks to find the hyperplane that maximizes the margin between the closest points of each class [1].

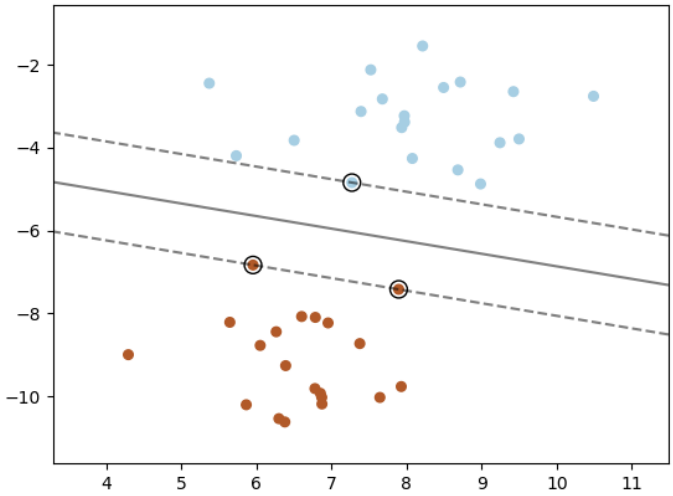


Figure 2 - Support Vector Machine

SVMs are particularly useful when the data is non-linearly separable, meaning that it is not possible to draw a straight line or plane to separate the data into classes. In such cases, SVMs use kernel functions to transform the data into a higher-dimensional space where it is linearly separable.

SVMs have been successfully applied to a variety of applications, including image classification, bioinformatics, and text classification.

1.4.2 Artificial Neural Network

An Artificial Neural Network (ANN) is a type of machine learning algorithm inspired by the structure and function of the human brain. ANNs consist of layers with a large number of interconnected processing nodes, called "neurons," which work together to solve complex problems.

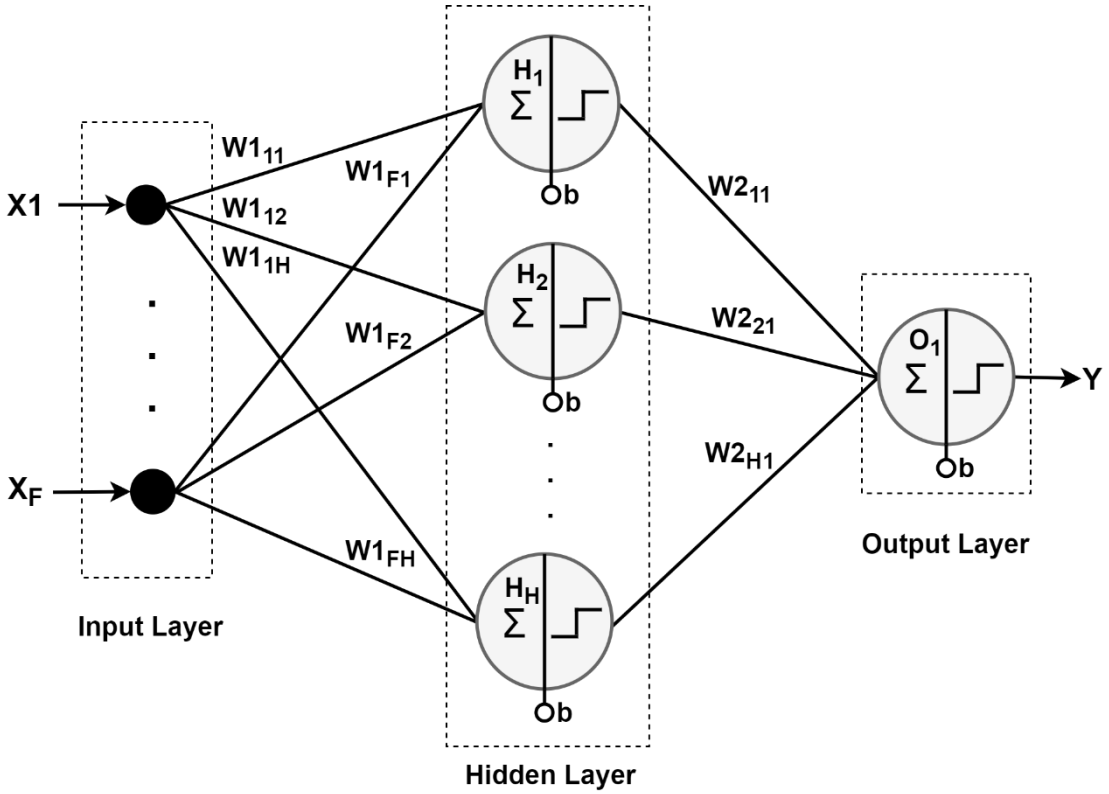


Figure 3 - Artificial neural network topology

Each neuron receives input from other neurons, processes that input, and then passes on its output to other neurons in the network. The neurons are organized into layers, with each layer performing a specific type of processing.

The input layer receives the data, and the output layer produces the desired output. The hidden layers in between perform the processing required to transform the input data into the desired output.

During the training phase, the weights of the connections between the neurons ($W_{ij} : i - layer, j - neuron\ number$) is adjusted through a process called backpropagation, where the errors between the predicted output and the actual output are propagated backwards through the network, allowing the weights to be updated to improve the accuracy of the model.

ANNs have been successfully applied to a wide range of applications, including image and speech recognition, natural language processing, and predictive analytics.

1.4.3 Decision Tree

A Decision Tree (DT) is an algorithm used in machine learning that uses a tree-like structure to represent a set of decisions and their possible consequences. It is a popular method used both for classification and regression analysis.

In a DT, each internal node represents a test on an attribute, each branch represents the outcome of the test, and each leaf node represents a class label or a numerical value. The tree is built recursively, with each split being chosen to maximize the information gain or minimize the impurity of the resulting subsets.

During the training phase, the algorithm selects the attributes that best separate the data into different classes or produce the best fit. The resulting tree can be used to make predictions on new data by following the path from the root node to a leaf node, which corresponds to the predicted class or value.

DTs are popular because they are easy to understand and interpret, can handle both categorical and numerical data, and can handle missing values. However, they can suffer from overfitting if the tree is too complex, and they can be sensitive to small variations in the data. Various techniques, such as pruning, can be used to prevent overfitting and improve the performance of Decision Trees.

1.4.4 Bagging

Bootstrap Aggregation (Bagging - BG) is a machine learning technique that involves combining multiple models (ensemble) to improve the accuracy and stability of predictions. BG is a type of ensemble learning, which combines the output of multiple models to produce a final prediction.

In BG, multiple models are trained independently, each on a different subset randomly sampled with replacement from the original dataset and the final prediction is obtained by averaging the predictions of all the models.

BG can be used with any type of model, but it is most commonly used with DTs, as they are prone to overfitting and thus can benefit from the additional variance introduced by the bootstrapping process. Bagging can improve the accuracy of the model and reduce the variance of the predictions.

1.4.5 Random Forest

Random Forest (RF) is a type of ensemble learning algorithm that combines multiple decision trees trained on different subsets of the data using Bootstrap Aggregation (Bagging).

In RF, the decision trees are constructed by randomly selecting a subset of features at each node, which reduces the correlation between the trees and improves their accuracy and stability. The final prediction is obtained by averaging the predictions of all the trees in the forest.

RF are popular because it can handle both categorical and numerical data, can handle missing values, and can perform well on high-dimensional datasets. They are also robust to noise and outliers and can detect feature interactions that may be missed by other algorithms.

RF can be used for both classification and regression tasks and has been successfully applied to a wide range of applications, including image and speech recognition, natural language processing, and predictive analytics. They are relatively easy to use and require minimal tuning, making them a popular go-to solution in the machine-learning world.

2. Weka [1]

Weka (which stands for Waikato Environment for Knowledge Analysis) is a software package that contains a collection of tools and algorithms for data analysis and predictive modelling [2].

2.1 Key features

Among a myriad of data science software packages, WEKA has gained a solid position in the academia and enterprise environment due to key features such as:

- it enables fast testing and prototyping of ML algorithms;
- it supports a wide range of standard ML tasks;
- it has a decent GUI, with a smooth learning curve that enables quick and effective entry into the world of data processing, visualization, and ML algorithms;
- the user can test the algorithms on its datasets and in a significant set of built-in/existing datasets, resulting from contributions from the international community.
- it can be used not only from this graphical user interface (GUI), but also from the command line (useful to automate data processing), or using its Java API (to automate, add Weka features to other software or develop new features/models);
- it is bundled with a wide range of ML algorithms and new ones can be added (via GUI or programming interface);
- it enables access to widely known ML toolboxes, such as R and scikit-learn;
- it is licensed under the GNU General Public License;

While WEKA's GUI takes some time to master, it provides a very comfortable environment to experiment with, from the simplest to very complex ML algorithms. Note that despite the wide portfolio of algorithms already available, when a specific

[1] To learn the basics of Weka, refer to the previous lab script: "Weka - A data science tool".

algorithm is not available, people are welcome to implement it and afterward freely distribute it to the community.

2.2 Setup

Given that it is implemented in the Java programming language, installing Weka is a very straightforward process, as follows:

- In Microsoft Windows, just download the software package and install it.
- In Linux, just download and extract the software zip archive, enter the newly extracted directory and run: `./weka.sh`
- In macOS, download and open the `.dmg file`, copy the contents to a directory of your choice and open the `weka-3-8-6-azul-zulu-osx` file.

Any additional info about the installation procedure can be found here.

3. Tutorial example

In this tutorial example, we are going through the steps of a classification problem using the Weka Explorer application to show the technical aspects and also the machine learning concepts involved such as supervised learning, training, validation and test data, cross-validation, the use of performance measures to evaluate the effectiveness of a classifier in its task.

In the **Weka Explorer** application and open the **iris dataset**. Now go to the **Classifier tab** and press the **Choose button** and under functions select the **LibSVM algorithm** [3]. Use a 10-fold cross-validation scheme to maximize our available data and train and test the model on this dataset.

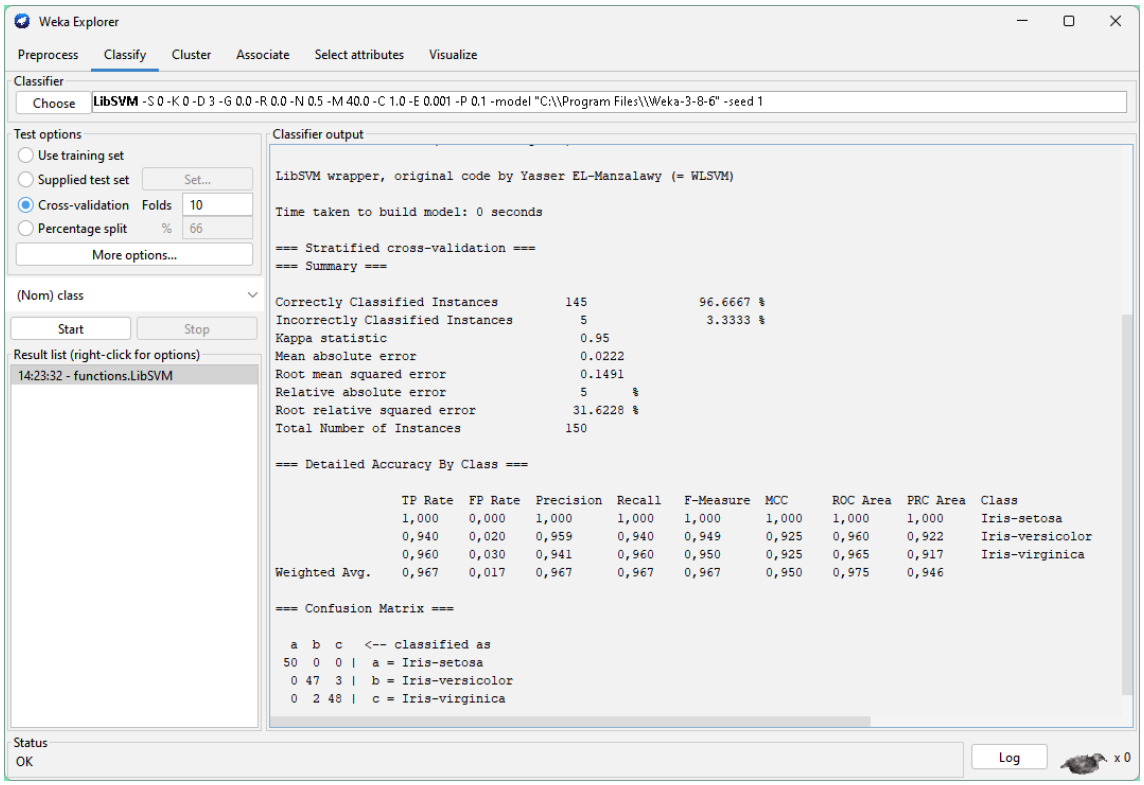


Figure 4 – LibSVM results on the iris dataset

The default output options used by Weka[2] print information about the train and test results:

The number of `Correct Classified Instances` was 145 which corresponds to a classification accuracy of 96.6667% with 5 `Incorrect Classified Instances` that correspond to a classification fail rate of 3.3333%. Note that 150 instances were classified since the cross-validation process was used, Otherwise, only a subset would be used to test our model (using the `Percentage split` option on the `Test options` box).

Afterward, the `Detailed Accuracy By Class` displays a set of performance metrics obtained in the test dataset, present in table format for each class and for the complete test set (in the last line). This way each user can access the model performance using the performance metrics to which is accustomed.

The measures are the: True Positive Rate (Sensitivity), False Positive Rate ($\text{FPR} = 1 - \text{TNR} = 1 - \text{Specificity}$), Precision, Recall, F-Measure, MCC, and ROC Area (under the curve).

The True Positive Rate is the Sensitivity which is a measure of the model's ability to detect positive cases. For the `iris-setosa` the TP Rate is 1.0 (100%) which

[2] more available at the `More options ...` button in the `Test options` box.

corresponds to the correct detection of all test data instances. For the `iris-versicolor` the sensitivity is 94% and for the `iris-virginica` 96%. The global mean test sensitivity 10 folds were 96.7%.

The fall-out or False Positive Rate is complementary to the specificity ($\text{FPR} = 1 - \text{TNR} = 1 - \text{Specificity}$). This means that the global FPR of 1.7% corresponds to a global test specificity of $100 - 1.7 = 98.3\%$.

Another very useful performance measure to compare classifiers is the ROC Area a.k.a. ROC-AUC. The Receiver Operating Characteristic (ROC) is a graphical representation of the curve formed by plotting TPR vs FPR and the Area Under the Curve (AUC) is the integral of that curve.

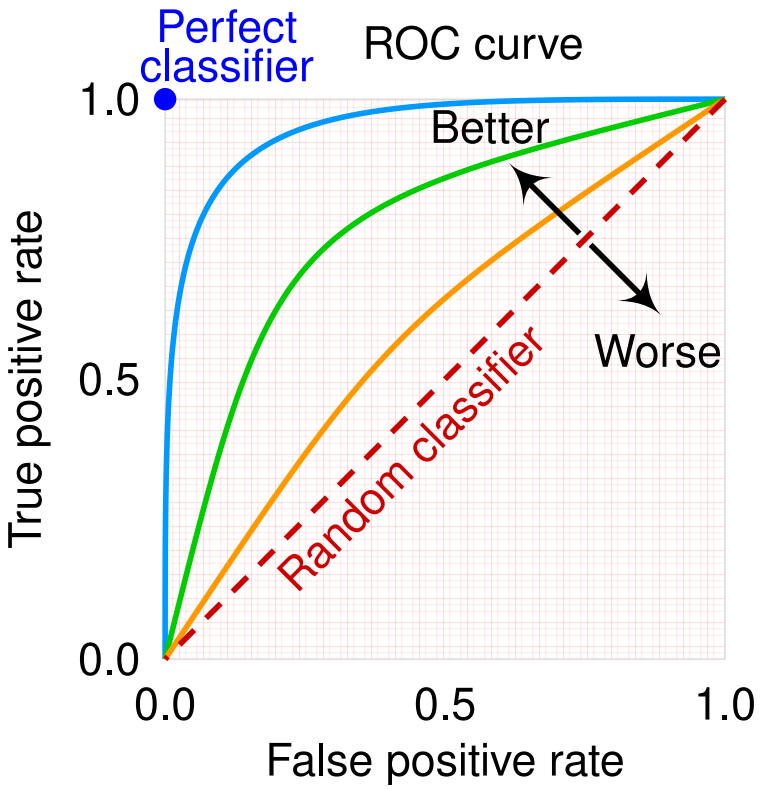


Figure 5 - ROC Curve

As depicted in Figure 5 a ROC-AUC of 0.5 corresponds to a random classifier and a ROC-AUC of 1 to the perfect classifier.

As expected the ROC-AUC is 1 for the `iris-setosa`, 0.96 for the `iris-versicolor`, 0.95 for the `iris-virginica` and 0.975 for all classes.

By default, the `LibSVM` uses a kernel with radial basis functions (available in the model parameters as `kernelType`) which is suited for datasets with non-linear frontiers. In the iris dataset, there are two types of frontiers. The linear one between the `iris setosa` and `iris versicolor` and the non-linear one between the `iris versicolor` and `iris virginica`. Change the `kernelType` to a linear one and then to a polynomial one and check the differences in the model performance in both cases.

4. Exercises

4.1 Problem A

Repeat the experiment and analysis done in tutorial example using an artificial neural network, decision tree, bagging and random forest classifiers and compare the results obtained for the three models. Access not only the performance metrics but also the training times.

Inspect each model's outputs and try to understand the decision process for each model. Did you manage to comprehend any of them? Which one? _____

_____

_____

_____

_____

_____

_____

Inspect its decision process a try to understand it.

4.2 Problem B

Go to the OpenML or Kaggle Datasets websites, select a dataset of your interest, study and explore it using the Weka Explorer `Preprocess` and `Select attributes` tabs, and afterward build and test classification models using the SVN, ANN and DT. Let's inspect and discuss the results in the PL class.

5. Challenge

Open the Weka Experimenter a set-up an experience to automate the comparison of these and other machine-learning models on the dataset that you selected in Problem B.

6. References

[1] A. J. Smola and B. Schölkopf, 'A tutorial on support vector regression', *Stat. Comput.*, vol. 14, pp. 199–222, 2004.

[2] The University of Waikato, 'Weka 3: Machine Learning Software in Java'. 2015. [Online]. Available: http://www.cs.waikato.ac.nz/ml/weka/

[3] C.-C. Chang and C.-J. Lin, 'LIBSVM: a library for support vector machines', *ACM Trans. Intell. Syst. Technol. TIST*, vol. 2, no. 3, pp. 1–27, 2011.